

Лабораторная работа №13

Программирование в командном процессоре ОС UNIX

Ветвления и циклы

Студент: Хайдаров Олим Фирдавсович

Группа: НБИбд-02-25

ORCID: 0000-0002-0877-7063

Дата: 2026



Оглавление

1. Цель работы
2. Выполнение лабораторной работы
 - 2.0 Использование getopts и grep
 - 2.1 Взаимодействие Си и Bash
 - 2.2 Циклы и управление файлами
 - 2.3 Архивация с помощью find
3. Вывод
4. Контрольные вопросы
5. Список литературы



Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать сложные командные файлы

Основные задачи:

- Использование логических управляющих конструкций
- Работа с циклами (`for` , `while` , `until`)
- Синтаксический анализ командной строки (`getopts`)
- Взаимодействие Bash и компилируемых программ

Задание 1: getopt + grep

Условие

Написать командный файл, который:

- Анализирует командную строку с ключами: `-i` , `-o` , `-p` , `-C` , `-n`
- Ищет в указанном файле нужные строки с помощью `grep`

Параметры:

- `-i` — входной файл
- `-o` — выходной файл
- `-p` — шаблон поиска
- `-C` — учитывать регистр
- `-n` — показывать номера строк

Задание 1: Создание файла

Создаём файл `task1_search.sh`

![Создание файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-31-31.png)

Задание 1: Код скрипта (части 1, 2, 3)

Пишем код Командного файла:

![Командный файл, 1-я часть](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-50-26.png)

![Командный файл, 2-я часть](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-50-56.png)

![Командный файл, 3-я часть](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-51-05.png)

- создаем тестовый файл - в нем будет набор строк с информацией, фрагмент которой мы будем в дальнейшем искать (помимо искомой, мы добавим строки с содержанием другой информации, для проверки работы программы) - это слово "Apple"

![Тестовый файл](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-52-29.png)

Далее мы делаем файл task1_search.sh исполняемым и запускаем процесс проверки работы:

![Проверка работы](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-53-31.png)

Также если мы введем файл без искомого слова, которое мы ищем и без пути, то нам выдается следующее:

![Тест файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-54-26.png)

После выдется опции параметров ввода -- снизу сообщения об ошибки -- с целью, чтобы пользователь специфицировал запрос на основе предложенных вариантов ввода

Также мы далее смотрим, что было записано в реестр-файл, записывающий результаты экзекуции программы:

![Реестр-файл результата](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 16-57-43.png)

Результат выполнения:

Скрипт корректно распознает флаги. Если передать ключ -С, поиск становится чувствительным к регистру. Если передать -п, в результат добавляются номера строк.

2.1 Взаимодействие Си и Bash {#21-взаимодействие-си-и-bash}

Условие: Написать программу на Си, которая определяет знак числа и завершается с кодом 0, 1 или 2. Написать скрипт, который вызывает эту программу и анализирует код возврата через \$?.

Создаем файл:

![Создание файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-07-51.png)

Пишем код программы, которая проверяет вводимые числа:

![Программа проверки чисел](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-05-48.png)

Далее код для компиляция Си-программы:

![Создание файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-47-03.png)

2-я часть

![Си-программа](/home/ofkhayjdardovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-42-58.png)

Далее делаем ее исполняемой через команду `chmod +x task2_wrapper.sh` (из первого скрина)

Запуск программы:

![Запуск программы](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-47-48.png)

-- проверяем, ввода Положительное, Отрицательное числа, и в конце вводим число 0 -- на что программа успешно отвечает нам и определила их соответствующе

Результат выполнения:

Скрипт компилирует .с файл, запускает его, считывает введенное число и выводит вердикт на основе кода завершения программы Си.

2.2 Циклы и управление файлами {#22-циклы-и-управление-файлами}

Условие: Написать файл, создающий N файлов (1.tmp...N.tmp) по аргументу. Скрипт должен уметь также и удалять эти файлы.

Создаем файл:

![Создание файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-52-57.png)

Потом вводим программу, которая создает N файлом и удаляет также эти файлы:

![Создание программы](/home/ofkhayjdardovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-52-05.png)

![Создание программы](/home/ofkhayjdardovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-52-12.png)

Делаем её исполняемой с помощью команды `chmod +x` (из первого скрина)

Запускаем и проверяем работу файла - успешно было создано 5 файлов

![Запуск работы файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-53-28.png)

Проверяем их наличие с помощью команды ls

![Проверка наличия файлов](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-53-59.png)

-- как видим, да, файлы имеются

Далее удаляем их

![Удаление файлов](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-54-31.png)

Программа работает, мы успешно создали и удалили 5 файлов .tmp - формата

Результат

Результат выполнения:

Запуск `./task3_files.sh create 5` создает файлы `1.tmp...5.tmp`. Запуск `./task3_files.sh delete 5` удаляет их.

2.3 Архивация с помощью find {#23-архивация-с-помощью-find}

Условие: Написать файл, который архивирует файлы, измененные менее недели назад, используя find и tar.

Создаем файл [Создание файла] :

![Создание файла](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-59-10.png)

Далее пишем код программы, соответствующей условиям:

![Код программы, 1-я часть](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-58-28.png
)

![Код программы, 2-я часть](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-58-53.png
)

Далее делаем файл исполняемым [Файл исполняемый] (с помощью команды `chmod +x`):

![Файл исполняемый](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 17-59-50.png)

Создаем файлы-реестры, с давностью изменения 2 дня назад, сегодня, 10 дней назад и 3 дня назад [Файлы-реестры] :

![Файлы-реестры](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 18-02-54.png)

Запуск работы и [Результат] :

![Результат](/home/ofkhayjdarovdk5n18/Pictures/Screenshots/Screenshot From 2026-05-09 18-03-10.png)

Результат выполнения:

Скрипт находит файлы (например, recent.txt, созданный вчера) и добавляет их в week_backup.tar.gz. Файлы, созданные месяц назад, игнорируются.

Выводы {#выводы}

В ходе лабораторной работы №13 я научился и изучил:

1. Синтаксический анализ командной строки с помощью команды `getopts` и реализовал поиск через `grep`;
2. Изучил механизм взаимодействия интерпретатора `Bash` и компилируемых программ (Си) через код завершения `$?`;
3. Научился использовать циклы `for` для пакетного создания и удаления файлов ;
4. Применил утилиту `find` с условием по времени изменения (`-mtime`) для выборочной архивации данных.

Контрольные вопросы {#контрольные-вопросы}

1. Каково предназначение команды getopt?

getopt предназначена для синтаксического анализа командной строки. Она позволяет скрипту корректно обрабатывать флаги (опции) и их аргументы, упрощая создание профессиональных утилит.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (*, ?, []) используются командным процессором для "разворачивания" (globbing) шаблонов в списки существующих имён файлов.

Например, *.txt генерирует список всех текстовых файлов в директории.

3. Какие операторы управления действиями вы знаете?

Операторы:

Последовательности: ;

Логические: && (И), || (ИЛИ)

Конвейер: |

Условные: if, case

Циклы: for, while, until

4. Какие операторы используются для прерывания цикла?

`break` — полностью прекращает выполнение цикла и выходит из него.

`continue` — прерывает текущую итерацию цикла и переходит к следующей проверке условия.

5. Для чего нужны команды false и true?

Эти команды используются для создания условий в циклах и проверках:

true всегда возвращает код успеха (0).

false всегда возвращает код ошибки (1).

Они часто используются в бесконечных циклах: while true; do ...; done.

6. Что означает строка `if test -f mans/i.$s`?

Эта строка проверяет существование обычного файла по пути `mans/i.$s`.

Оператор `test -f` возвращает "истину" (0), если файл существует и является обычным файлом (не директорией).

7. Объясните различия между конструкциями `while` и `until`.

`while` выполняет тело цикла пока условие истинно (код возврата 0).

`until` выполняет тело цикла пока условие ложно (код возврата не 0), т.е. пока условие не станет истинным.

Список литературы {#список-литературы}

Список литературы

1. Кулябов Д. С. и др. Операционные системы: учебное пособие. — М.: РУДН, 2024. — С. 97–98.
2. GNU Bash Manual. Free Software Foundation, 2026. — URL: <https://www.gnu.org/software/bash/manual/>



Спасибо за внимание!

Вопросы?

Контакты:



1032242468@rudn.ru



ORCID: 0000-0002-0877-7063



Группа: НБИбд-02-25